

Project final de programmation C

2023-24

TD : 4

TP : 6

Membres : Cui Xiang, Cao Yunliang

Répartition des tâches

- Cui Xiang :

Principalement responsable de la mise en œuvre simple du jeu dans le terminal, en se concentrant sur le mouvement des unités individuelles, la production de nouvelles unités dans la colonie, les combats entre les unités et l'accès aux fichiers du jeu.

- Cao Yunliang :

Principalement responsable de l'intégration et de l'implémentation graphique du jeu, il s'est concentré sur la construction réussie de colonies multiples et la gestion des bugs, ainsi que sur l'application de la MLV.

pièce jointe

- code source(final_project_CUI Xiang_CAO Yunliang.c)
- rapport général(Project_final_de_programmation_C_CUI Xiang_CAO Yunliang.pdf)

Instructions pour l'utilisation

1.

Compilation : les compilateurs gcc et clang peuvent compiler nos fichiers avec succès, mais n'oubliez pas d'ajouter le suffixe "-IMLV" pour exécuter la bibliothèque MLV lorsque vous entrez dans la commande de compilation.

Exemple :

```
gcc test.c -o project_final -IMLV
```

2.

système applicable : système d'exploitation Linux (debian, ubuntu).

3.

Ligne de commande : lorsque vous souhaitez exécuter un exécutable compilé, il existe deux façons.

- ◆ Saisissez directement le nom du fichier pour accéder au mode de jeu normal.

Exemple :

```
PS C:\Users\26451\Desktop\C> .\project_final.exe
```

- ◆ Entrez le nom du fichier immédiatement suivi du nom de l'archive pour entrer dans le mode de lecture de l'archive du jeu.

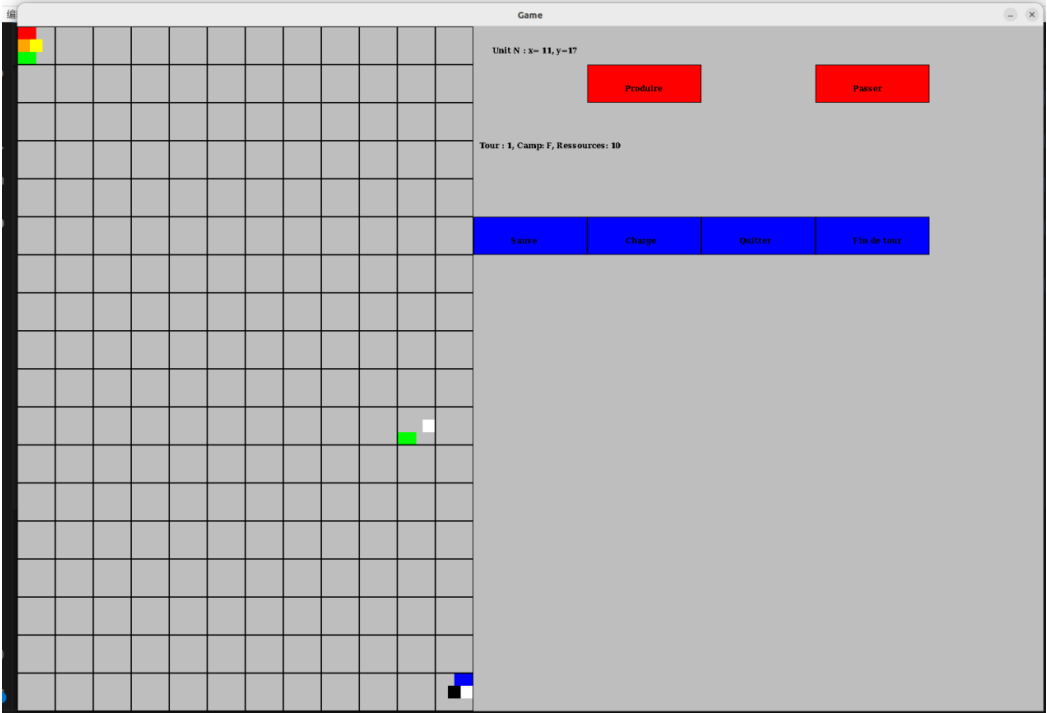
Exemple:

```
PS C:\Users\26451\Desktop\C> .\project_final.exe project_final_save
```

4.

Dans l'interface graphique : vous devez maintenir le bouton gauche de la souris enfoncé, déplacer la souris vers un espace vide et relâcher le bouton lorsque l'interface change. Les carrés noirs dans l'image représentent les unités en action.

Exemple :



Partie réalisée

1. A terminé avec succès l'initialisation de la disposition du plateau et la création et le placement des camps et unités initiaux du premier plateau
2. A terminé la production de la ruche, déplacement de la reine, des ouvrières et des guerrières, et collecte du pollen par les ouvrières dans le camp des abeilles
3. A Terminé la production du nid, de la reine et le mouvement de la frelon et du reine.
4. A Terminé la partie où vous pouvez construire plusieurs ruches dans le camp des abeilles et le camp des frelons.
5. Vous pouvez archiver le jeu.
6. Des graphiques simples ont été réalisés en utilisant la bibliothèque graphique MLV
7. Nous pouvons placer plusieurs unités du même type sur la même caisse.

partie insatisfaisante

1. La section sur la lecture de l'archive du jeu afin de charger le jeu a rencontré de nombreux problèmes et n'a pas été achevée avec succès.
2. Pour la section sur la bataille entre les deux camps, une simple bataille entre les deux camps a été implémentée avec succès dans la section du terminal, mais des problèmes ont été rencontrés avec la bibliothèque MLV et elle n'a pas été achevée en raison de problèmes de temps.
3. Les escouades nouvellement générées ne peuvent pas être déplacées, ce qui est dû au fait que la fonction ***updateunitlists*** et la fonction ***placerunite*** ne sont pas assez parfaites.
4. L'absence de nombreuses indications d'erreur, par exemple, si vous sélectionnez une orientation immobile, vous ne pouvez pas vous déplacer sans que l'on vous rappelle que c'est parce que la direction sélectionnée ne peut pas être avancée.
5. Les trois types de mise à jour de la table à chaîne ne sont pas parfaits, jusqu'à présent la mise à jour de la table à chaîne comporte toujours des erreurs, ce qui reflète une compréhension insuffisante de la table à chaîne.

Introduction aux fonctions

1. *void printList(UListe list)*

Cette fonction est principalement utilisée pour éditer les camps des autres unités appartenant à la même ruche pour une unité sélectionnée,

afin qu'ils puissent être facilement visualisés et clairement répertoriés.

de manière à ce qu'ils puissent être facilement visualisés et clairement répertoriés.

L'argument est un pointeur struct sur une unité.

2. *int force(char type)*

Cette fonction est principalement utilisée pour renvoyer la valeur de la puissance pour chaque type d'unité.

Les arguments utilisent le type d'unité.

3. *Unite *createUnite(char camp, char type,int force)*

Cette fonction est principalement utilisée pour construire les unités initiales dans le tableau initial.

Les paramètres spécifient le camp pour lequel nous voulons construire l'unité, le type d'unité et la valeur de la puissance de l'unité.

4. *void placerUnite(Grille *g, Unite *u, int x, int y)*

Cette fonction est principalement utilisée pour placer les nouvelles unités que nous avons créées sur le tableau initial, et peut également être utilisée pour insérer un tableau de chaînes en V. Elle est pratique pour vérifier que les coordonnées du cas répondent aux exigences et qu'il n'y a pas déjà d'autres unités en conflit dans le même cas.

Elle est pratique pour vérifier que les coordonnées du cas répondent aux exigences et qu'il n'y a pas déjà d'autres unités en conflit dans le même cas.

Les paramètres spécifient le tableau à sélectionner, les unités à placer et les coordonnées auxquelles les unités doivent être placées.

5. *void updateUnitLists(Grille *g, Unite *u)*

Cette fonction est principalement utilisée pour mettre à jour la table de chaîne qu'elle affecte lorsqu'une nouvelle unité est créée.

Elle modifie principalement les deux tableaux de chaîne globaux abeille ou frelon et place toutes les unités dans l'ordre d'action.

Les paramètres spécifient le tableau sélectionné et les nouvelles unités que nous ajoutons.

6. ***int bouton(int x,int y,int buttonX,int buttonY,int buttonWidth,int buttonHeight)***

Cette fonction est principalement utilisée pour agir comme un bouton. Lorsque le joueur clique sur la zone correspondante, cette fonction détermine la portée effective du bouton pour lancer l'effet correspondant.

Les paramètres spécifient la position du clic de la souris et les limites d'un bouton pour déterminer si une certaine zone est cliquée.

7. ***Unite* prochaineUniteAbeille(Grille *g)***

Cette fonction est principalement utilisée pour passer à l'unité suivante dans le camp d'abeilles, en donnant la priorité aux unités du même type si elles existent encore. Lorsque toutes les unités du même type ont agi, elle laisse agir le type suivant jusqu'à la dernière unité.

Les paramètres spécifient le tableau sélectionné.

8. ***Unite* prochaineUniteFrelon(Grille *g)***

Cette fonction est principalement utilisée pour passer à l'unité suivante dans le camp des frelons, en donnant la priorité aux unités du même type si elles existent encore. Lorsque toutes les unités du même type ont agi, elle laisse agir le type suivant jusqu'à la dernière unité.

Les paramètres spécifient la carte sélectionnée.

9. ***void initialiserPlateau(Grille *g)***

Cette fonction est principalement utilisée pour initialiser le plateau de jeu initial.

Dans cette fonction, la fonction ***createUnite*** précédente est utilisée ainsi que la fonction ***updateUnitLists*** et la fonction ***placerUnite***.

Dans cette fonction, la fonction ***createUnite*** est utilisée pour créer les unités initiales du plateau, puis la fonction ***placerUnite*** est utilisée pour placer les unités nouvellement créées, et enfin la fonction ***updateUnitLists*** est utilisée pour mettre à jour les listes chaînées concernées.

Les paramètres spécifient le tableau que nous voulons initialiser.

10. **void supprimer_unite(Grille *g,char type,int posx,int posy,Unite *u,int recolte)**

Cette fonction permet de supprimer une unité décédée et de modifier la structure de la table liée concernée.

Dans cette fonction, les paramètres spécifient l'unité à supprimer, le type de l'unité, ses coordonnées actuelles et la raison de sa mort.

Tout d'abord, nous déterminons le type de l'unité à supprimer, et s'il s'agit d'une ruche, elle est supprimée directement.

Ensuite, s'il ne s'agit pas d'une ruche, nous déterminons si elle s'est autodétruite ou si elle a été tuée par l'ennemi.

C'est là que le paramètre recolte est utilisé. Si le paramètre recolte n'est pas 2 alors il s'autodétruit, si le paramètre reclote est 2 il est tué par l'ennemi, ce qui ajoute des ressources à l'ennemi.

Enfin, nous ajoutons des ressources en fonction de la valeur de l'unité supprimée.

11. **void debutTour(Grille *g)**

Cette fonction détermine la fin d'un tour par **fin**, et est principalement utilisée pour résumer la fin d'un tour.

Elle compte l'accumulation des ressources, le nombre de tours, le règlement des batailles, les informations et le temps de production, etc. Elle choisit ensuite au hasard le camp qui commencera le tour suivant, et remet finalement **fin** à zéro.

Les paramètres spécifient le plateau sélectionné.

12. **void afficherPlateau(Grille *g)**

Cette fonction est principalement utilisée pour afficher l'ensemble du tableau, qui est divisé en 12*18 avec 60*60 pixels par grille. Chaque grille est divisée en 7 parties, qui représentent les différentes unités. Pour ce faire, nous parcourons l'occupant et la colonie de chaque cas afin de dessiner facilement toutes les unités.

Les paramètres spécifient le tableau sélectionné.

13. *void Ordre(Unite *unit)*

Cette fonction est principalement utilisée pour afficher l'unité qui est en train d'agir, avec son nom et son emplacement, et pour afficher les actions prises en charge par cette unité, par exemple le déplacement, l'autodestruction.

L'argument spécifie l'unité qui est sur le point d'agir.

14. *void Menu(Grille *g)*

Cette fonction est principalement utilisée pour afficher les actions hors plateau que le joueur peut choisir, telles que sauvegarder, quitter, et des informations sur le camp, les unités produites et le temps restant.

Les paramètres précisent le plateau sélectionné

15. *int finDetour(Grille *g)*

Cette fonction est principalement utilisée pour déterminer si un tour est terminé ou non.

Si les deux joueurs ont déjà joué un coup dans le tour, la fonction **debutTour** est activée, si un seul joueur a joué un coup, les joueurs sont échangés et le numéro de jugement du tour **fin** est fixé à un.

Les paramètres spécifient le tableau sélectionné.

16. *int deplacement(Grille *g, Unite *unit)*

Cette fonction est principalement utilisée pour contrôler le mouvement des différentes unités.

Nous utilisons l'entrée de la souris pour déterminer la direction du mouvement de l'unité, puis nous déterminons si la position sélectionnée par l'utilisateur répond aux exigences du jeu.

Si ce n'est pas le cas, l'unité ne se déplace pas. Dans le cas contraire, les attributs de position de l'unité en mouvement sont modifiés, ainsi que les attributs de position dans la table de jeu.

Les paramètres indiquent quelles unités doivent être déplacées et quels tableaux de jeu doivent être modifiés.

17. *int produit_ruche(Grille *g, Unite *unit)*

Cette fonction est utilisée pour produire une nouvelle unité à partir d'une ruche.

Nous utilisons d'abord la souris pour déterminer les coordonnées de la nouvelle unité à produire, puis nous sélectionnons le type de nouvelle unité que nous

voulons produire, et nous déduisons ensuite les ressources correspondantes dans la ruche en fonction de la valeur de l'unité, et si les ressources ne sont pas suffisantes, la production échoue.

Modifiez également la propriété de production de l'unité pour indiquer qu'elle est en cours de production.

Enfin, la propriété *toursrestant* de l'unité est modifiée pour déterminer le nombre de tours restants dans la production.

Les paramètres spécifient le diagramme de jeu et le producteur que nous devons modifier.

18. *int produit_nid(Grille *g, Unite *unit)*

Cette fonction est principalement utilisée par le nid pour produire de nouvelles unités.

La logique de cette fonction est identique à celle de la fonction *produit_ruche*.

La différence est qu'elles consomment des ressources différentes pour produire de nouvelles unités ; les abeilles consomment du pollen tandis que les frelons consomment du ressource.

19. *void produit_reine(Grille *g, Unite *u)*

Cette fonction est principalement utilisée par la reine pour produire une nouvelle ruche.

Dans cette fonction, nous déterminons d'abord quel camp décide de produire une nouvelle ruche.

Ensuite, nous créons une nouvelle ruche aux coordonnées de la reine, déduisons les ressources appropriées et modifions la propriété *toursrestant* de *unite* pour déterminer le nombre de tours restants à produire.

Les paramètres spécifient le plateau que nous devons modifier et le producteur.

20. *void ouvriere_recolte(Grille *g, Unite *unit)*

Cette fonction est principalement utilisée pour permettre aux abeilles ouvrières du camp d'abeilles de choisir leur mode de collecte.

Dans cette fonction, les paramètres spécifient d'abord la carte qui doit être modifiée ainsi que l'unité qui effectuera la collecte de pollen.

La fonction détermine d'abord l'attribut de production de l'unité qui doit être en mode *RECLOTE*, puis le nombre total de tours nécessaires à l'unité pour

collecter et le nombre de tours restants.

21. **void save_game(Grille *g,char camp)**

Cette fonction est principalement utilisée pour enregistrer la progression du jeu.

Les paramètres spécifient d'abord le plateau de jeu actuel et le camp qui agira en premier lors du tour en cours.

La première ligne du fichier est déterminée en écrivant le premier joueur et le nombre de ressources restantes des deux côtés.

Ensuite, le programme itère à travers toutes les unités du tableau de jeu et écrit les données correspondantes pour chaque unité dans le fichier, ligne par ligne, dans l'ordre.

22. **void war_combat(Grille *g,int pos_x,int pos_y)**

Cette fonction est principalement utilisée pour exécuter le comportement de combat de Unite.

Dans cette fonction, le plateau de jeu actuel est d'abord spécifié ainsi que les coordonnées où la bataille a lieu.

Tout d'abord, nous utilisons **look_campunit_incise** pour itérer sur tous les cas du plateau, et s'il y a des unités de deux camps différents dans le même cas, alors nous nous battons pour nous assurer qu'il n'y a qu'une seule unité de chaque camp dans chaque cas.

Ensuite, en fonction de la valeur de puissance de chaque unité et du nombre aléatoire, il jugera quel camp est gagnant, et le camp perdant utilisera la fonction **supprimer_unite** pour supprimer l'unité perdante.

Jusqu'à ce que le plateau actuel soit traversé, il n'y a pas d'unités de camps différents dans le même cas.

23. **int gagner_ou_pas(Grille *g)**

Cette fonction est principalement utilisée pour déterminer si un tour est terminé et si quelqu'un a gagné.

Les arguments de cette fonction spécifient le plateau de jeu sur lequel le jeu se déroule actuellement.

La fonction parcourt toutes les cases du plateau de jeu pour déterminer s'il y a encore deux camps sur le plateau ; s'il n'y a qu'un seul camp, ce camp gagne et le jeu se termine. Dans le cas contraire, le jeu se poursuit.

24. **int look_campunit_incise(Grille *g, int posx,int posy)**

Cette fonction est principalement utilisée pour parcourir le contenu de la grille de jeu.

La fonction spécifie le plateau en cours de jeu et les coordonnées de la position que l'utilisateur souhaite parcourir.

Cette fonction est utilisée conjointement avec la fonction ***war_combat*** pour déterminer s'il y a des unités de différents camps dans chaque cas du jeu afin de décider s'il faut ou non passer en mode combat.

Exemple de jeu

1. partie terminale (un tour)

◆ Interface initiale du jeu

```
PS C:\Users\26451\Desktop\C> .\project_final.exe
welcome, game start !!!
Rrog - | - | - | - | - | - | - | - | - | - |
- | - | - | - | - | - | - | - | - | - |
- | - | - | - | - | - | - | - | - | - |
- | - | - | - | - | - | - | - | - | - |
- | - | - | - | - | - | - | - | - | - |
- | - | - | - | - | - | - | - | - | - |
- | - | - | - | - | - | - | - | - | - |
- | - | - | - | - | - | - | - | - | - |
- | - | - | - | - | - | - | - | - | -Nr|
Le jeu continue.
Le jeu continue.
=====Maintenant, c'est tour 1=====
le random nombre est 1
```

◆ Déterminer le camp avancé et demander si une sauvegarde est nécessaire

```
le random nombre est 1
-----
|Rrog - | | o| o| o| o| o| o| o| o| o| o|
-----
| o| o| o| o| o| o| o| o| o| o| o|
-----
| o| o| o| o| o| o| o| o| o| o| o|
-----
| o| o| o| o| o| o| o| o| o| o| o|
-----
| o| o| o| o| o| o| o| o| o| o| o|
-----
| o| o| o| o| o| o| o| o| o| o| o|
-----
| o| o| o| o| o| o| o| o| o| o| o|
-----
| o| o| o| o| o| o| o| o| o| o| o|
-----
| o| o| o| o| o| o| o| o| o| o| -Nr |
-----
tour de frelon!!!
ressource reste : 10
pouvez vous souhaitez save??? Y:1,N:0
```

◆ Demander à l'utilisateur ce qu'il doit faire ensuite

```
pouvez vous souhaitez save??? Y:1,N:0
0
Choix de la production de la NID en (9, 9).
1. Produire une reine(8 pollen, 8 tours).
2. Produire une Frelon(3 pollen, 5 tours).
3. Passer de tour.
4. Detruire l'unite.
```

- ◆ Exécute l'opération et affiche l'heure de fin de l'opération.

Choix de la production de la NID en (9, 9).

1. Produire une reine(8 pollen, 8 tours).
2. Produire une Frelon(3 pollen, 5 tours).
3. Passer de tour.
4. Detruire l'unite.

1

position : (9,9) ? ?

type:N,camp:F,posx:9,psy:9

-----1.colonie_temp est full-----

-----2.colonie_temp->usuiv est full-----

avant commande tempsrestant:0

apres commande tempsestant:0

avant commande tempsrestant:0

apres commande tempsestant:0

avant commande tempsrestant:0

apres commande tempsestant:0

Choix de la reine en (9, 9).

1. deplacer
2. construire une ruche.
3. passer le tour.

3

avant commande tempsrestant:9

subscribe_tours

apres commande tempsestant:8

- ◆ Le jeu n'est pas terminé, le choix de l'autre camp est demandé et un nouveau plateau est affiché

Le jeu continue.

Rrog -		o	o	o	o	o	o	o	o	o	o
	o	o	o	o	o	o	o	o	o	o	o
	o	o	o	o	o	o	o	o	o	o	o
	o	o	o	o	o	o	o	o	o	o	o
	o	o	o	o	o	o	o	o	o	o	o
	o	o	o	o	o	o	o	o	o	o	o
	o	o	o	o	o	o	o	o	o	o	o
	o	o	o	o	o	o	o	o	o	o	o
	o	o	o	o	o	o	o	o	o	o	o
	o	o	o	o	o	o	o	o	o	o	o
	o	o	o	o	o	o	o	o	o	o	-Nr

tour de abeille!!!

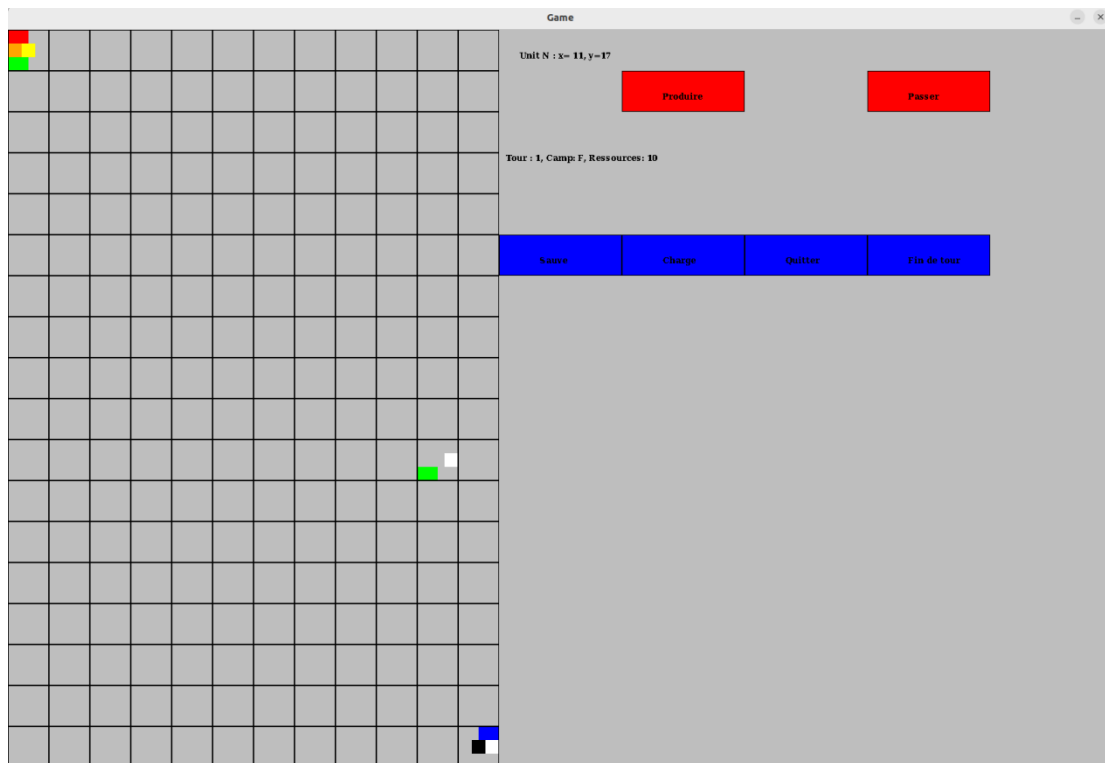
pollen reste : 10

Choix de la production de la ruche en (0, 0).

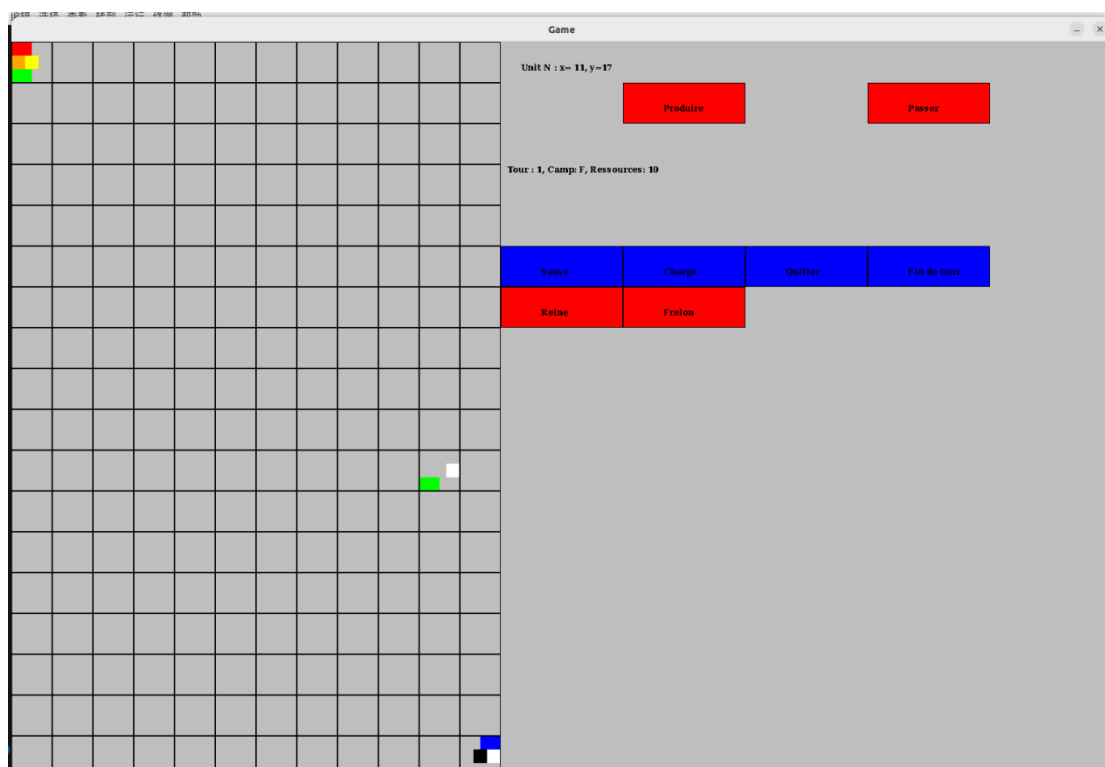
1. Produire une reine(7 pollen, 8 tours).
2. Produire une Ouvriere(3 pollen, 2 tours).
3. Produire une Guerriere(5 pollen, 4 tours).
4. Produire une escadron(6 pollen, 6 tours).
5. Passer de tour.
6. Detruire une unite.

2. Composants graphiques

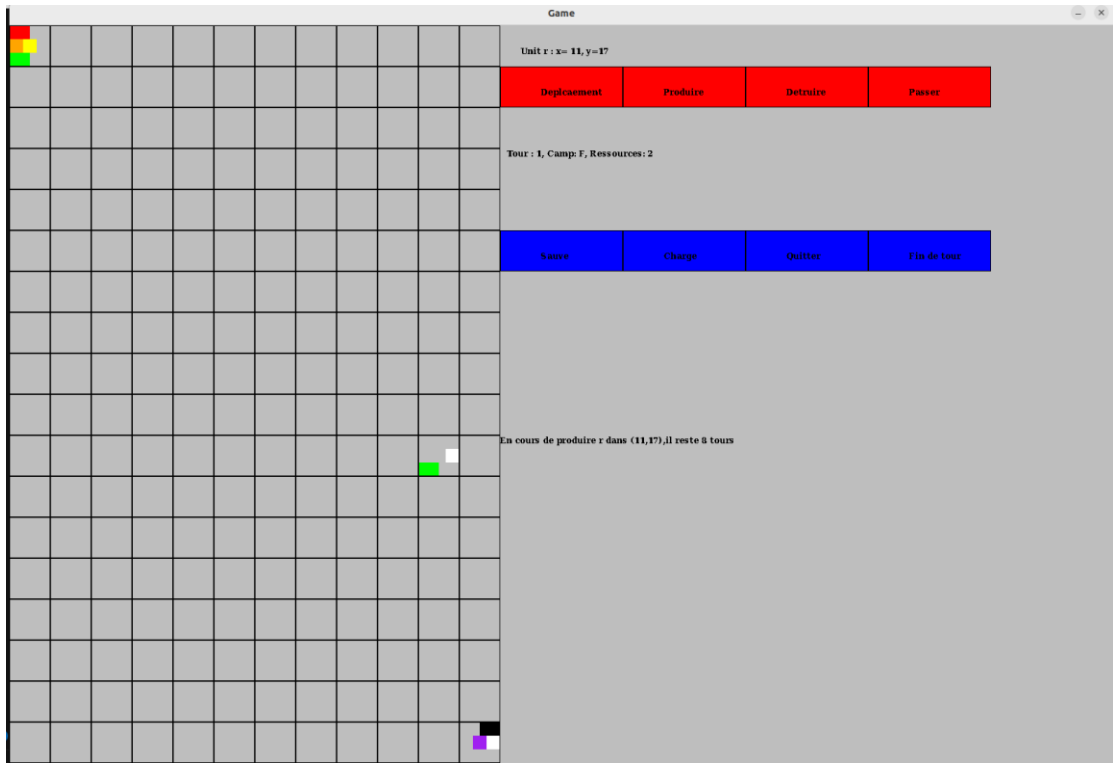
- Début de la partie : vous pouvez savoir quel camp commence en premier en utilisant le texte ou les carrés noirs



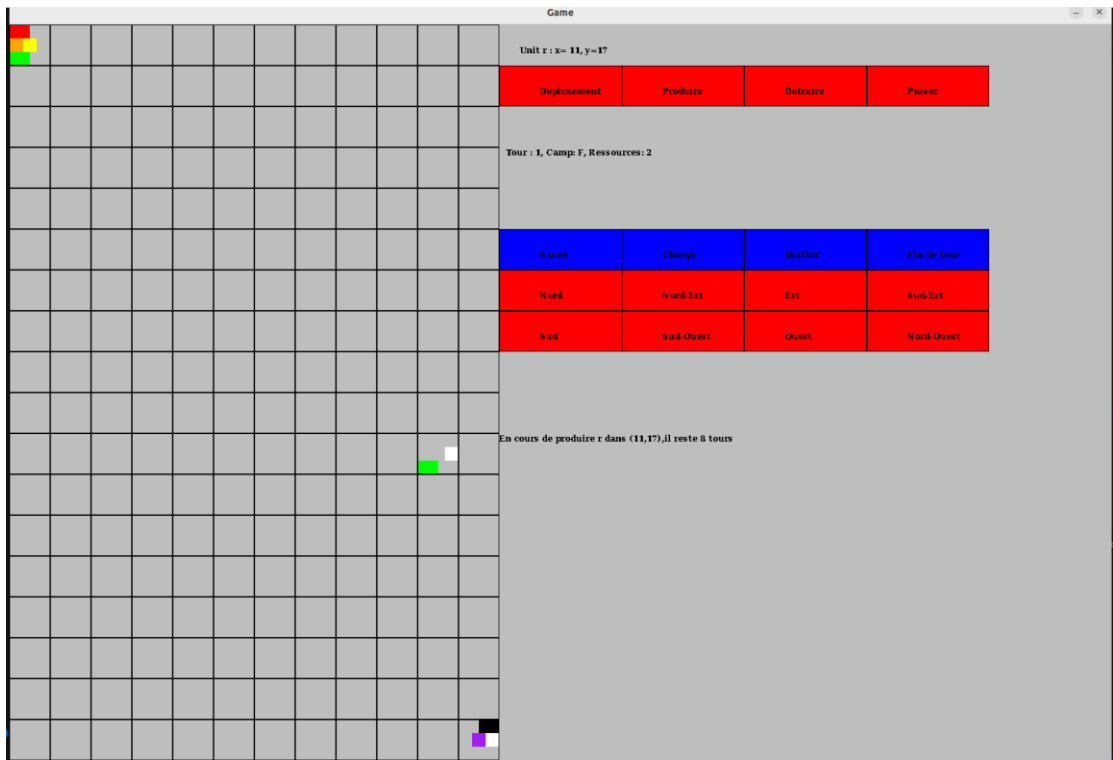
- Vous pouvez sélectionner le carré supérieur sur lequel agir (en prenant l'exemple de la production)

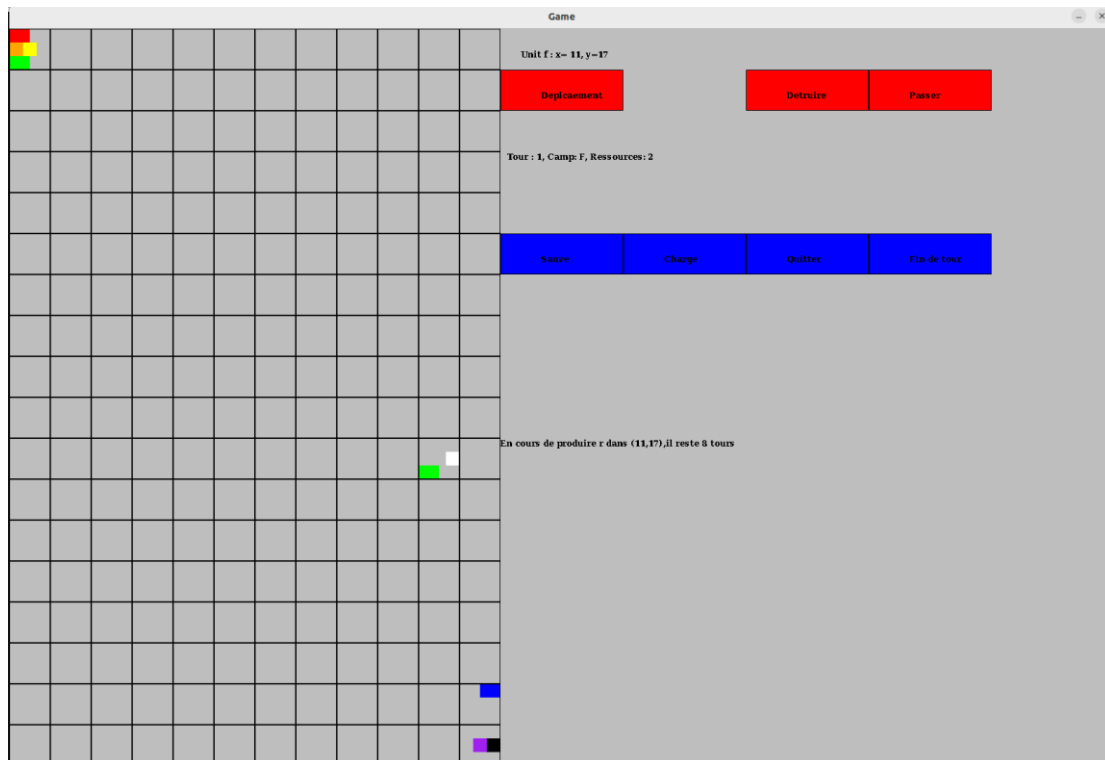


- Après la sélection, vous verrez les informations correspondantes et passerez automatiquement à l'unité d'action suivante

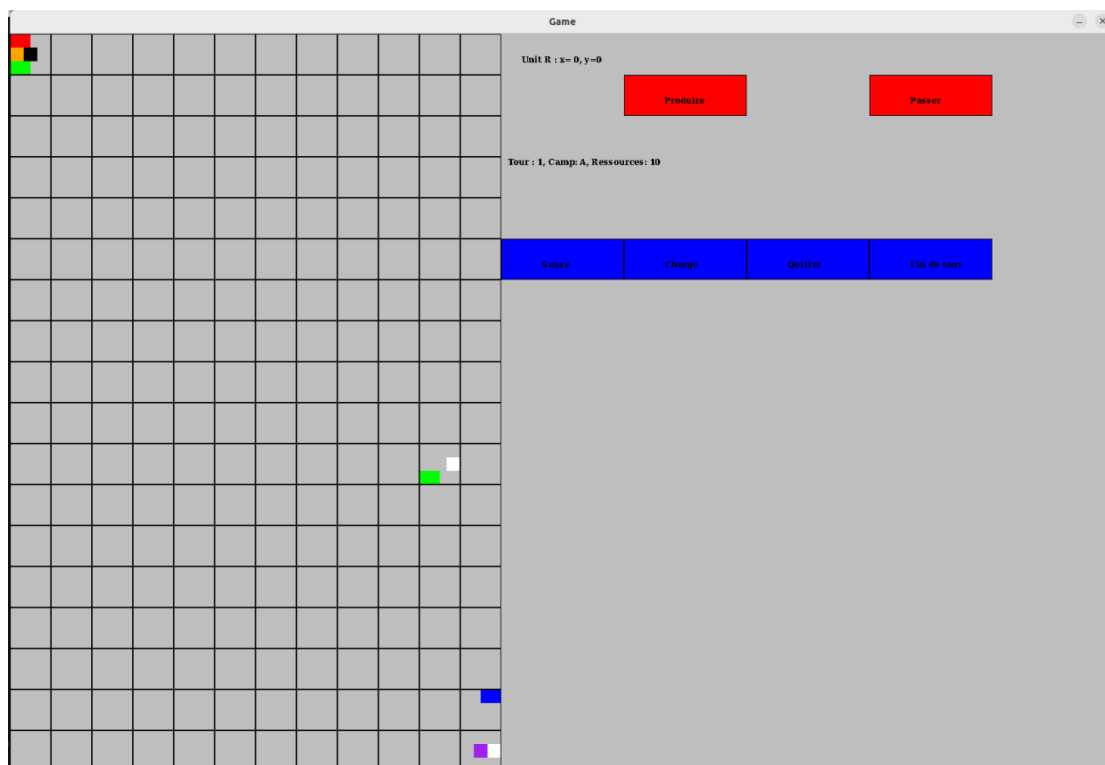


- Lorsque l'option Déplacer est sélectionnée, les cases du diagramme se déplacent en fonction de la direction du mouvement





- Lorsque tout le monde a fini d'agir (vous pouvez compter sur le fait de cliquer sur le **passueur** pour confirmer cela, s'il ne saute pas c'est que tout le monde a agi) si vous voulez terminer le tour de soi plus tôt, cliquez sur **fin de tour** pour changer de côté.



- Si vous souhaitez enregistrer, cliquez sur le bouton Enregistrer et le fichier sera automatiquement enregistré



projet_final

- Cliquez sur Exit pour quitter.